

NAME: Atchaya Vharsne S

REG. NO.: 192524185

COURSE CODE: CSA0401

COURSE NAME: OPERATING SYSTEMS

CO5 AT2 Pseudocode Writing Task (Algorithm Design)

1. File System Space Management Algorithm

BEGIN

SET threshold = 80

usage = GET_DISK_USAGE()

IF usage > threshold THEN

IDENTIFY all log_files

SORT log_files BY size DESCENDING

FOR each file IN log_files DO

IF file is important THEN

COMPRESS or ROTATE file

ELSE

DELETE file

ENDIF

usage = GET_DISK_USAGE()

IF usage <= threshold THEN

BREAK

ENDIF

ENDFOR

ENDIF

END

Working

1. The algorithm first checks the current disk utilization.
2. If disk usage is 80% or less, no action is taken.
3. If usage exceeds 80%, all log files are identified.
4. The files are sorted in descending order of size, so larger files are handled first.
5. Old or unnecessary logs are deleted, while important logs can be compressed or rotated.
6. After each operation, disk usage is checked again.
7. The process stops once disk utilization falls to 80% or below.

Result: This algorithm automatically prevents disk overflow by monitoring disk space and managing log files efficiently.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int threshold = 80; 5 int usage; 6 int n, i; 7 int size[100]; 8 9 printf("Enter current disk usage percentage: "); 10 scanf("%d", &usage); 11 12 if (usage > threshold) { 13 14 printf("Disk usage exceeded %d%%\n", threshold); 15 16 printf("Enter number of log files: "); 17 scanf("%d", &n); 18 19 printf("Enter sizes of log files:\n"); 20 21 for (i = 0; i < n; i++) { 22 printf("Log file %d size: ", i + 1); 23 scanf("%d", &size[i]); 24 } 25 26 for (i = 0; i < n - 1; i++) { 27 for (int j = i + 1; j < n; j++) { 28 if (size[i] < size[j]) { 29 int temp = size[i]; 30 size[i] = size[j]; 31 size[j] = temp; 32 } 33 } 34 } 35 36 for (i = 0; i < n; i++) { 37 if (usage <= threshold) 38 break;</pre>		<pre>Enter current disk usage percentage: 92 Disk usage exceeded 80%! Enter number of log files: 3 Enter sizes of log files: Log file 1 size: 500 Log file 2 size: 200 Log file 3 size: 700 Deleting/Rotating log file of size 700 MB Current disk usage: 87% Deleting/Rotating log file of size 500 MB Current disk usage: 82% Deleting/Rotating log file of size 200 MB Current disk usage: 77% Disk usage is now under control: 77% === Code Execution Successful ===</pre>

2. Priority-Based CPU Scheduling Algorithm

BEGIN

INPUT number_of_processes

FOR each process DO

 INPUT Process_ID

 INPUT Priority

 INPUT Burst_Time

 SET Remaining_Time = Burst_Time

ENDFOR

WHILE there are incomplete processes DO

 SORT ready_queue by Priority in descending order

 SELECT process with highest Priority

 ALLOCATE CPU for one time quantum

 EXECUTE selected process

 DECREASE Remaining_Time

 IF Remaining_Time = 0 THEN

 MARK process as Completed

 REMOVE process from ready_queue

 ELSE

 REDUCE Priority slightly

 ADD process back to ready_queue

 ENDIF

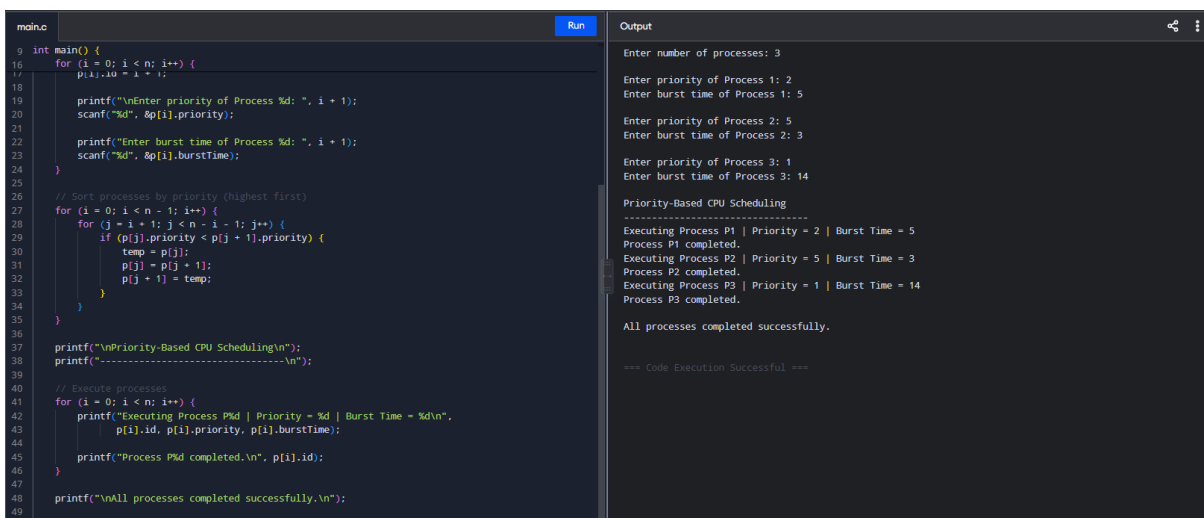
ENDWHILE

DISPLAY "All processes completed"

END

Working

1. The algorithm accepts a list of processes with their priority levels.
2. The processes are sorted in descending order of priority.
3. The process with the highest priority is selected first.
4. CPU time is allocated to the selected process.
5. The process is executed.
6. If the process is completed, it is removed from the process list.
7. If it is not completed, its priority is slightly reduced and it is added back to the process list.
8. The process list is sorted again based on priority.
9. The algorithm continues until all processes are completed.



```
main.c Run Output
9 int main() {
16     for (i = 0; i < n; i++) {
17         p[i].id = i + 1;
18
19         printf("\nEnter priority of Process %d: ", i + 1);
20         scanf("%d", &p[i].priority);
21
22         printf("Enter burst time of Process %d: ", i + 1);
23         scanf("%d", &p[i].burstTime);
24     }
25
26     // Sort processes by priority (highest first)
27     for (i = 0; i < n - 1; i++) {
28         for (j = i + 1; j < n - i - 1; j++) {
29             if (p[j].priority < p[j + 1].priority) {
30                 temp = p[j];
31                 p[j] = p[j + 1];
32                 p[j + 1] = temp;
33             }
34         }
35     }
36
37     printf("\nPriority-Based CPU Scheduling\n");
38     printf("-----\n");
39
40     // Execute processes
41     for (i = 0; i < n; i++) {
42         printf("Executing Process P%d | Priority = %d | Burst Time = %d\n",
43               p[i].id, p[i].priority, p[i].burstTime);
44
45         printf("Process P%d completed.\n", p[i].id);
46     }
47
48     printf("\nAll processes completed successfully.\n");
49 }
```

Enter number of processes: 3

Enter priority of Process 1: 2
Enter burst time of Process 1: 5

Enter priority of Process 2: 5
Enter burst time of Process 2: 3

Enter priority of Process 3: 1
Enter burst time of Process 3: 14

Priority-Based CPU Scheduling

Executing Process P1 | Priority = 2 | Burst Time = 5
Process P1 completed.
Executing Process P2 | Priority = 5 | Burst Time = 3
Process P2 completed.
Executing Process P3 | Priority = 1 | Burst Time = 14
Process P3 completed.
All processes completed successfully.
=== Code Execution Successful ===

Result: The CPU gives preference to high-priority processes while slightly reducing the priority of incomplete processes to avoid starvation and improve overall CPU efficiency.

3. RAM Allocation for Multiple Virtual Machines

```
BEGIN

SET total_RAM = 16 GB

SET host_reserved = 4 GB

SET available_RAM = total_RAM - host_reserved

INPUT number_of_VMs

SET min_RAM = 2 GB

FOR i = 1 TO number_of_VMs DO

    IF available_RAM >= min_RAM THEN

        ALLOCATE min_RAM to VM[i]

        available_RAM = available_RAM - min_RAM

        PRINT "RAM allocated to VM", i

    ELSE

        PRINT "Insufficient memory for VM", i

    ENDIF

ENDFOR

PRINT "Remaining available RAM =", available_RAM

END
```

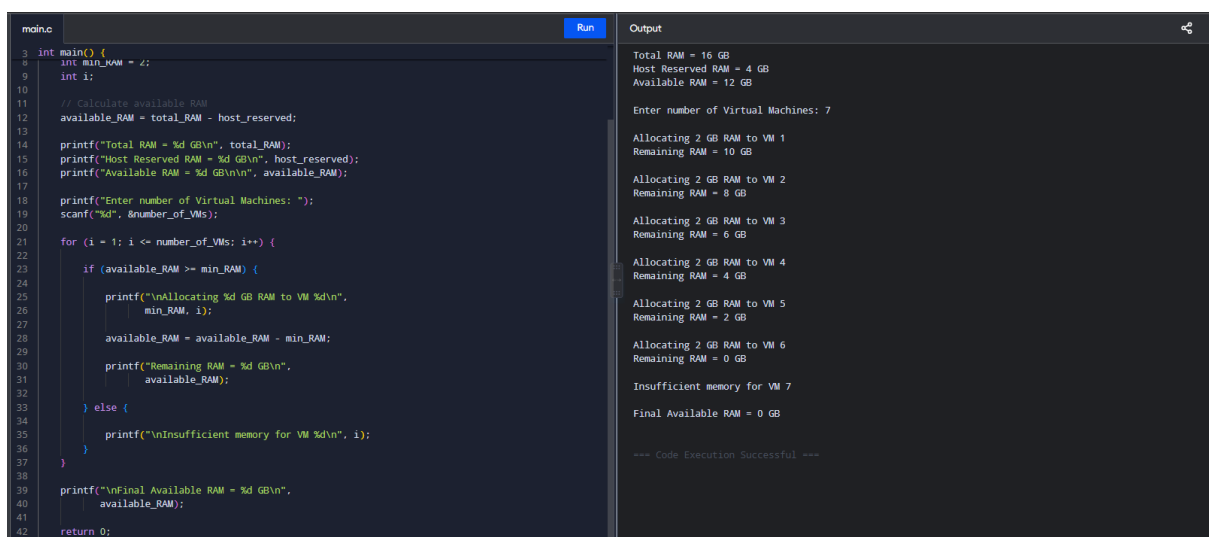
Working

1. The system has a total RAM of 16 GB.
2. 4 GB is reserved for the host operating system.
3. Therefore, the available RAM for virtual machines is:

Available RAM = 16 GB - 4 GB = 12 GB

4. The number of virtual machines is taken as input.
5. Each VM requires a minimum of 2 GB RAM.
6. Before allocating RAM, the algorithm checks whether enough memory is available.
7. If at least 2 GB is available, RAM is allocated to the VM and the available RAM is reduced.
8. If sufficient memory is not available, the system displays "Insufficient memory".
9. Finally, the remaining RAM is displayed.

Result: This algorithm efficiently distributes available RAM among multiple virtual machines while ensuring that the host system always retains its reserved memory.



```
main.c Run Output
3 int main() {
4     int min_RAM = 2;
5     int i;
6
7     // Calculate available RAM
8     available_RAM = total_RAM - host_reserved;
9
10    printf("Total RAM = %d GB\n", total_RAM);
11    printf("Host Reserved RAM = %d GB\n", host_reserved);
12    printf("Available RAM = %d GB\n", available_RAM);
13
14    printf("Enter number of Virtual Machines: ");
15    scanf("%d", &number_of_VMs);
16
17    for (i = 1; i <= number_of_VMs; i++) {
18        if (available_RAM >= min_RAM) {
19            printf("\nAllocating %d GB RAM to VM %d\n",
20                  min_RAM, i);
21            available_RAM = available_RAM - min_RAM;
22            printf("Remaining RAM = %d GB\n",
23                  available_RAM);
24        } else {
25            printf("\nInsufficient memory for VM %d\n", i);
26        }
27    }
28
29    printf("\nFinal Available RAM = %d GB\n",
30          available_RAM);
31    return 0;
32 }
```

Output

```
Total RAM = 16 GB
Host Reserved RAM = 4 GB
Available RAM = 12 GB

Enter number of Virtual Machines: 7

Allocating 2 GB RAM to VM 1
Remaining RAM = 10 GB

Allocating 2 GB RAM to VM 2
Remaining RAM = 8 GB

Allocating 2 GB RAM to VM 3
Remaining RAM = 6 GB

Allocating 2 GB RAM to VM 4
Remaining RAM = 4 GB

Allocating 2 GB RAM to VM 5
Remaining RAM = 2 GB

Allocating 2 GB RAM to VM 6
Remaining RAM = 0 GB

Insufficient memory for VM 7

Final Available RAM = 0 GB

=== Code Execution Successful ===
```

4. Hypervisor Selection Algorithm

```
BEGIN

    INPUT performance_required

    INPUT budget

    INPUT hardware_access

    IF performance_required = HIGH AND hardware_access = TRUE THEN

        SELECT Type_1_Hypervisor

    ELSE IF budget = LOW THEN

        SELECT Type_2_Hypervisor

    ELSE

        SELECT Type_1_Hypervisor

    ENDIF

    PRINT selected_hypervisor

END
```

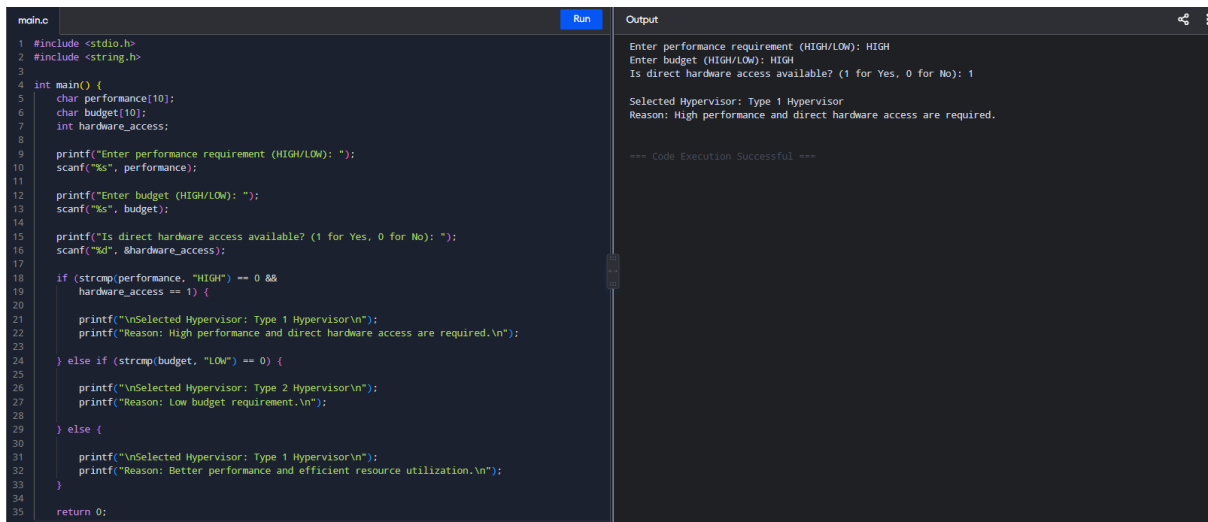
Working

1. The algorithm takes three inputs:
2. Performance requirement
3. Available budget
4. Hardware access requirement
5. If high performance is required and direct hardware access is available, the algorithm selects a Type 1 Hypervisor.
6. If the budget is low, it selects a Type 2 Hypervisor, which is generally suitable for smaller-scale or less demanding environments.
7. In all other cases, the algorithm selects a Type 1 Hypervisor for better resource utilization and performance.

Result

Type 1 Hypervisor: Selected for high-performance environments and efficient resource utilization.

Type 2 Hypervisor: Selected when cost or budget constraints are the main concern.



```
main.c Run Output
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char performance[10];
6     char budget[10];
7     int hardware_access;
8
9     printf("Enter performance requirement (HIGH/LOW): ");
10    scanf("%s", performance);
11
12    printf("Enter budget (HIGH/LOW): ");
13    scanf("%s", budget);
14
15    printf("Is direct hardware access available? (1 for Yes, 0 for No): ");
16    scanf("%d", &hardware_access);
17
18    if (strcmp(performance, "HIGH") == 0 &&
19        hardware_access == 1) {
20
21        printf("\nSelected Hypervisor: Type 1 Hypervisor\n");
22        printf("Reason: High performance and direct hardware access are required.\n");
23
24    } else if (strcmp(budget, "LOW") == 0) {
25
26        printf("\nSelected Hypervisor: Type 2 Hypervisor\n");
27        printf("Reason: Low budget requirement.\n");
28
29    } else {
30
31        printf("\nSelected Hypervisor: Type 1 Hypervisor\n");
32        printf("Reason: Better performance and efficient resource utilization.\n");
33    }
34
35    return 0;
36 }
```

Enter performance requirement (HIGH/LOW): HIGH
Enter budget (HIGH/LOW): HIGH
Is direct hardware access available? (1 for Yes, 0 for No): 1

Selected Hypervisor: Type 1 Hypervisor
Reason: High performance and direct hardware access are required.

=== Code Execution Successful ===

5. Resource Limiting Using CPU and Memory Controls

BEGIN

INPUT application_list

FOR each app IN application_list DO

CREATE cgroup for app

SET CPU_limit for app

SET memory_limit for app

ASSIGN app process to its cgroup

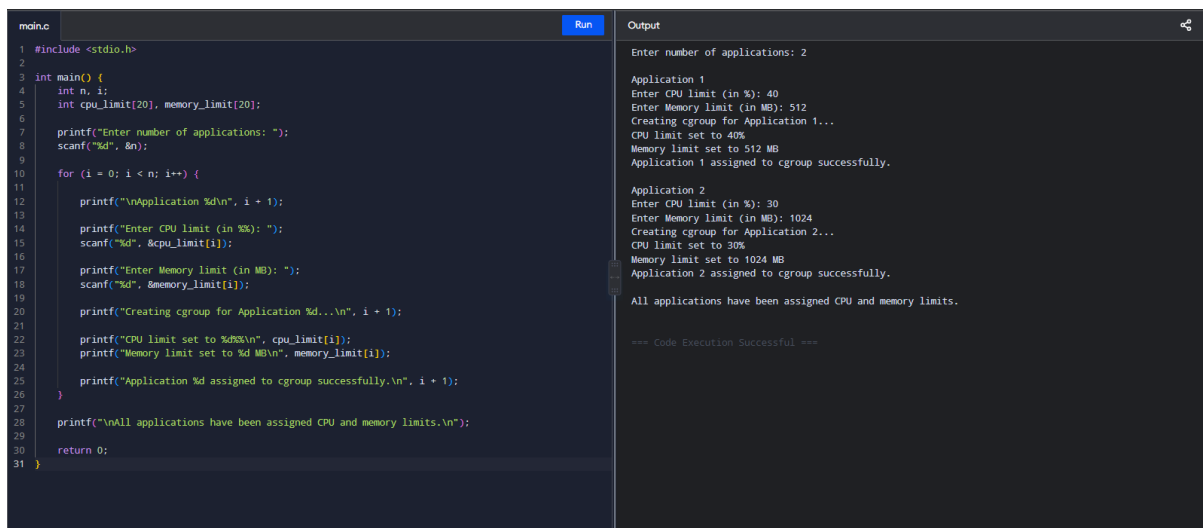
ENDFOR

PRINT "CPU and memory limits applied successfully"

END

Working

1. The algorithm takes a list of applications as input.
2. For each application, a separate group is created.
3. A maximum CPU usage limit is assigned.
4. A maximum memory usage limit is assigned.
5. The application's process is assigned to its respective group.
6. This ensures that no single application consumes excessive CPU or memory resources.



```
main.c Run Output
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     int cpu_limit[20], memory_limit[20];
6
7     printf("Enter number of applications: ");
8     scanf("%d", &n);
9
10    for (i = 0; i < n; i++) {
11
12        printf("\nApplication %d\n", i + 1);
13
14        printf("Enter CPU limit (in %): ");
15        scanf("%d", &cpu_limit[i]);
16
17        printf("Enter Memory limit (in MB): ");
18        scanf("%d", &memory_limit[i]);
19
20        printf("Creating cgroup for Application %d...\n", i + 1);
21
22        printf("CPU limit set to %d%%\n", cpu_limit[i]);
23        printf("Memory limit set to %d MB\n", memory_limit[i]);
24
25        printf("Application %d assigned to cgroup successfully.\n", i + 1);
26    }
27
28    printf("\nAll applications have been assigned CPU and memory limits.\n");
29
30    return 0;
31 }
```

Enter number of applications: 2

Application 1

Enter CPU limit (in %): 40

Enter Memory limit (in MB): 512

Creating cgroup for Application 1...

CPU limit set to 40%

Memory limit set to 512 MB

Application 1 assigned to cgroup successfully.

Application 2

Enter CPU limit (in %): 30

Enter Memory limit (in MB): 1024

Creating cgroup for Application 2...

CPU limit set to 30%

Memory limit set to 1024 MB

Application 2 assigned to cgroup successfully.

All applications have been assigned CPU and memory limits.

=== Code Execution Successful ===

Result: The system maintains balanced resource utilization and improves overall performance by controlling CPU and memory usage of each application.